

Lab 2: Solving Linear Systems with Numpy and Scipy

CAS CS 132: Geometric Algorithms

Due October 8, 2026 by 8:00PM

Outline. You'll be benchmarking NumPy's and SciPy's linear system solvers. In particular, you'll be looking at how differently structured matrices affect the running time of these solvers. You're required to submit a write-up for this lab as a PDF on Gradescope. The details of what you're required to submit are given in the last section below.

Introduction

Both NumPy and SciPy include functions for computing exact solutions to well-determined linear systems: `numpy.linalg.solve`¹ and `scipy.linalg.solve`². Both functions take as arguments an $(n \times n)$ matrix `a` and a vector `b` in $\mathbb{R}^n$. In particular, the matrix `a` must be square and the equation `a @ x = b` must have a unique solution. Otherwise, both functions throw a `numpy.linalg.LinAlgError` and report that the matrix is “singular.”³ You should spend some time looking at the documentation of each of these functions.

The implementations of both of these functions are somewhat complicated. Most of the work is done by calls to LAPACK⁴ subroutines, which in turn call BLAS⁵ subroutines. These low-level subroutines are highly optimized for the hardware on which the code is running. SciPy's solver has an additional optimization: it analyzes the input matrix for special structure, and then dispatches one of several solving backends that are optimized for said structure. We'll be benchmarking NumPy's and SciPy's solvers on different matrix structures to see how this affects running time.

Much of doing linear algebra with computers is about using domain-specific knowledge about the underlying structure of the matrices of interest to speed up relevant matrix computations. For example, adjacency matrices from networks

¹<https://numpy.org/doc/stable/reference/generated/numpy.linalg.solve.html>

²<https://docs.scipy.org/doc/scipy/reference/generated/scipy.linalg.solve.html#scipy.linalg.solve>

³We'll discuss singular matrices and invertibility in a couple weeks.

⁴<https://netlib.org/lapack/>

⁵<https://netlib.org/blas/>

are often sparse, i.e., have very few nonzero entries. If the network is undirected, then it's also symmetric. If the network is a grid, then it's banded, and often tridiagonal. We'll be looking at the latter two special classes of matrices mentioned above.

1. A square matrix is **symmetric** if its entries below the diagonal are the same as those above. That is, $A_{ij} = A_{ji}$ for all i and j satisfying $1 \leq i, j \leq n$. For example,

$$\begin{bmatrix} 1 & 2 & 3 \\ 2 & 4 & 5 \\ 3 & 5 & 6 \end{bmatrix}$$

is a symmetric matrix.

2. A square matrix is **tridiagonal** if its nonzero entries are either on, just above, or just below the diagonal. That is, A_{ij} is 0 if $|i - j| \geq 2$. For example,

$$\begin{bmatrix} 1 & 2 & 0 & 0 & 0 \\ 3 & 4 & 5 & 0 & 0 \\ 0 & 6 & 7 & 8 & 0 \\ 0 & 0 & 9 & 10 & 11 \\ 0 & 0 & 0 & 12 & 13 \end{bmatrix}$$

is a tridiagonal matrix.

It's not difficult to imagine why tridiagonal matrices are easier to solve; think about how many row operations you'd have to do using Gaussian elimination. That it should be easier to solve symmetric systems is less obvious, and sometimes doesn't turn out to be the case in practice.

Lab Write-Up

This section contains the tasks you're required to complete for this lab. Some questions require you to write code, others require you to reason about code. If a question is labeled with a “(*)”, it's the kind of question that could appear on a quiz or on an exam.

1. (*) The fact that NumPy's and SciPy's solvers don't work on rectangular matrices might seem a bit limiting at first glance. Suppose we wanted to use these solvers on rectangular matrices. Our idea is to pad the given coefficient matrix with zeros until it becomes square. If the matrix has more rows than columns, then we pad with zeros on the right. If it has more columns than rows, we pad on the bottom with zeros; this also requires padding the right-hand-side vector with zeros. Implement the function `pad_with_zeros` in the starter code.

Include your implementation (just the code, not the docstring) in your write-up.

2. (*) Explain why the idea in the previous problem will not work. In particular, what will happen if we call `numpy.linalg.solve` or `scipy.linalg.solve` on the output of `pad_with_zeros`?

Include your explanation in your write-up.

3. Run the included code for benchmarking the functions `numpy.linalg.solve` and `scipy.linalg.solve` against uniformly random matrices. You may have to change the input value depending on how long the code takes (you should expect it to take 5 to 20 minutes). The output graph should show a visible separation in the curves.

Include the output graph in your write-up. Describe in 1-2 sentences what the graph tells you.

4. (*) Implement the function `random_tridiagonal` which constructs an `n` by `n` tridiagonal matrix with entries chosen uniformly at random from the range `low` to `high`, together with an `n` element random vector with entries from the same range (you can use `random_gen` as a template).

Include your implementation (just the code, not the docstring) in your write-up.

5. Run the included code for benchmarking the functions `numpy.linalg.solve` and `scipy.linalg.solve` against uniformly random tridiagonal matrices. Again, you may have to change the input value depending on how long the code takes. Again, the output graph should show a visible separation in the curves.

Include the output graph in your write-up. Describe in 1-2 sentences what the graph tells you.

6. (*) Implement the function `random_sym` which constructs an `n` by `n` symmetric matrix with entries chosen uniformly at random from the range `low` to `high`, together with an `n` element random vector with entries from the same range.

Include your implementation (just the code, not the docstring) in your write-up.

7. Run the included code for benchmarking the functions `numpy.linalg.solve` and `scipy.linalg.solve` against uniformly random symmetric matrices.

Include the output graph in your write-up. Describe in 1-2 sentences what the graph tells you.

NumPy Cheatsheet

After this lab, you should be familiar with the following NumPy concepts.

- `numpy.linalg.solve` and `scipy.linalg.solve`
- `numpy.random.Generator.uniform`
- `numpy.diag`
- `numpy.tril` and `numpy.triu`